

Low-Rate Quantization of a Hybrid Language Model: What Moves Loss, and Where the Bytes Actually Are

Shayan Hemmatiyan
Independent Researcher

September 23, 2026

Controlled post-training quantization study on one 0.8B and one 27B hybrid linear-attention checkpoint.
No quantization-aware training, no serving stack, and no claim of a new algorithm.

Abstract

We report a controlled study of post-training weight quantization below two bits in a hybrid linear-attention language model, asking which factors move validation loss under storage measured from bytes actually written. Three findings are worth more than the codec comparisons that motivated the study. First, **byte allocation dominates codec design at this scale**: quantizing the tied embedding costs 0.000098 NLL per 100 MB saved against 0.020873 for the first step of the weight codec and 2.127315 for its last, so a model with 2.792-bit weights and an INT4 embedding is half the size and takes a fifth of the damage of one with ternary weights and a BF16 embedding. Every sub-two-bit configuration we measure is off the efficient frontier for whole-model size — a statement about parameter shares, since the embedding is 33.8% of this checkpoint but 9.15% of the 27B one. Second, **the calibration distribution is a lever that costs no bytes**: damage is minimized when calibration and evaluation domains match, and a prespecified 50/50 mixture at the same token budget cuts worst-domain damage below either pure condition over three disjoint draws (0.095211 against 0.147013 and 0.191296), with a half-budget control showing this is coverage of activation directions rather than sample count. Third, **the codec results hold but are smaller than the above**: a shared eight-dimensional codebook beats a learned three-level scalar code by 0.036118 NLL [0.033768, 0.038414] at 0.43% more payload bytes on 0.8B DeltaNet projections and keeps its sign under eight refits; GPTQ-style compensation removes 68–69% of the damage, roughly three times the codec’s contribution, while *raising* plain weight MSE; index rate then dominates compensation and coverage dominates rate, yet a Hessian-derived per-tensor ranking built to exploit coverage is worse than random selection. The direction reproduces on a 27B checkpoint, but eight refits there span -0.007049 to $+0.000185$ and the effect is smaller than the spread refitting induces ($0.46\times$ the refit range against $5.05\times$ at 0.8B), so we report it as a direction on average and lean on it nowhere. Finally, the converted model is **served on compressed weights**: all 186 non-embedding matrices replaced by code-and-codebook modules reproduce the decoded-BF16 loss to 1.6×10^{-5} at $1.92\times$ less resident memory by the allocator, and $3.0\times$ cuBLAS latency on an isolated projection once the required Hadamard rotation is fused. We claim no new algorithm, and we separate what is established from what is not.

1 Scope and contributions

Vector quantization with Hessian-aware compensation, and Hadamard incoherence processing with eight-dimensional codes, are established ideas [1, 2, 3, 4]. We do not claim a new algorithm; the block extension of GPTQ in Section 5 is a routine generalization and we treat it as implementation,

not contribution. What we offer is controlled evidence, with explicit boundaries, about a setting the literature covers thinly: rates below two bits, on the recurrent projections of a hybrid model, with storage measured from bytes that are actually written and weights that are actually read back.

Concretely, and in order of how much they move a deployment decision rather than of how much work they took:

1. **Byte allocation dominates codec design at this scale** (Section 8). Crossing the weight-rate ladder with four embedding precisions, the marginal cost of a saved byte differs by four orders of magnitude between the two axes, and a configuration with higher-rate weights and a coarser embedding dominates every sub-two-bit configuration in both size and loss. The embedding quantizer we use is deliberately naive, which makes the comparison conservative against the codec. The conclusion follows from parameter shares and therefore inverts with scale, which we state quantitatively rather than leaving as a caveat.
2. **Calibration is the lever that costs nothing** (Section 7). Damage is minimized when calibration and evaluation domains match, and a prespecified 50/50 mixture at the same token budget reduces worst-domain damage below either pure condition on three disjoint draws; a half-budget control separates domain coverage from sample count. Unlike dimension, rate and coverage, this is paid for in neither bytes nor calibration tokens. We give the identity that explains why mismatch has an effect and why weight MSE cannot see it.
3. **The converted model is served on compressed weights** (Section 8), reproducing its decoded-BF16 loss to 1.6×10^{-5} at a resident footprint measured from the allocator, which is below what payload arithmetic predicts because the served index layout is wider than the stored one.
4. **Dimension.** A shared 6561-point eight-dimensional codebook beats a learned three-level scalar code at near-equal payload bytes, on two tensor families of a 0.8B model and on the QKV projections of a 27B model (Table 2). Eight dimensions earn their place at 0.8B, where the margin over four keeps its sign in all eight refits (0.005101 to 0.018548). At 27B the frozen run gives 0.004034 (CI [0.002715, 0.005334]), but that is a single refit and we did not measure this contrast across refits there; given that the larger VQ8-against-scalar contrast varies six-fold over three 27B refits, the 27B dimensionality margin should be treated as unreplicated. A 512-block pilot had suggested the opposite ordering, with four dimensions capturing 95.4% of the gain; that estimate did not survive the full stream.
5. **Compensation dominates the codec.** Error compensation is worth 68–69% of the damage against the codec’s 6–32%, and it improves loss while degrading weight MSE.
6. **Rate dominates in turn**, falling exponentially with no knee over the measured interval (Table 6).
7. **Coverage dominates all of them**, and the obvious way to exploit that — ranking tensors by activation-weighted error per byte — is worse than random.
8. **Two negative results on proxies and one on method.** Weight MSE and activation-weighted error each predict loss inside the regime where they are fitted and fail outside it; a self-consistent calibration loop is unnecessary because the dependency graph is acyclic.
9. **Robustness of the main result.** On these 0.8B DeltaNet projections the eight-dimensional code’s advantage survives refitting over calibration draws and seeds, held-out text, a second

domain, and context extension through 2048 tokens. Its magnitude varies across those conditions, and finer scalar grouping shows no consistent benefit in any of them.

2 Related work and what is new here

Every technique this paper uses is established. Hessian-weighted error compensation is GPTQ [3] and its asymmetric refinement [6]; incoherence processing by random Hadamard rotation is QuIP [4] and QuIP# [2], and QuaRot [7] applies the same idea to activations for 4-bit inference; multi-dimensional codebooks are GPTVQ [1] and, in additive form, AQLM [5]. Our block extension of GPTQ to vector codes (Section 5) is a routine generalization that collapses to textbook scalar GPTQ at dimension one, which we verify bit-exactly; we treat it as implementation.

What we contribute is controlled evidence in a setting the literature covers thinly, and three results that the literature’s framing tends to hide.

The architecture. Published low-rate results concentrate on dense attention transformers. The recurrent projections of a hybrid linear-attention model have a different error-propagation structure — a quantized `in_proj_qkv` perturbs a recurrent state rather than a one-shot attention map — and we find effects there that a transformer study would not surface, including damage that depends on position within a block.

Calibration sensitivity is usually a robustness check; here it is the headline. Williams and Aletras [8] show that calibration choice materially changes post-training quantization and pruning outcomes. We take that further in one specific direction: because the calibration set costs no stored bytes, its effect is not a confound to be controlled but a free parameter to be optimized, and we show a prespecified fixed-budget mixture beats every pure condition on worst-domain damage.

Byte accounting for the whole model, not the covered tensors. Low-rate papers report bits per weight over the tensors they convert. That number is the right one for comparing codecs and the wrong one for sizing a model. Measuring the whole checkpoint — and then measuring the served model from the allocator — is what surfaces the result that at this scale the embedding, not the codec, governs the byte budget.

3 What this paper does not establish

We separate these from the results because earlier drafts of this work did not, and the reader should not have to reconstruct the boundary.

- **No general claim that embeddings should be quantized before weights.** Our byte-allocation result is measured on one 0.8B checkpoint, with one naive embedding quantizer, on one corpus. It follows from the embedding’s 33.8% parameter share, and we show that share is 9.15% on the 27B checkpoint, where the same arithmetic points the other way — but we did not measure embedding quantization at 27B, so the inversion is an argument about bytes, not a result. Nor did we try rotation or error compensation on the embedding, so its ladder is a floor rather than a characterization.
- **No serving stack.** A full-coverage conversion is served on compressed weights and measured from the allocator (§8), but the embedding, norms, KV cache and recurrent state stay BF16, it runs on one GPU at batch one, and both kernels are tuned only far enough to be correct.
- **No latency claim.** A compressed matmul is verified and measured (§8), and it is $3.0\times$ *slower* than a BF16 cuBLAS product at batch one. The memory reduction is real; a speed advantage is not established, and neither kernel is tuned beyond correctness.

Run	target params	share	eval targets	calib. tokens
ternary_task_v4	0	0.0%	261,284	65,536
mlp_task_v1	264,241,152	35.1%	261,284	–
mixed_alloc_v1	377,487,360	50.2%	261,284	–
rate_curve_v2	377,487,360	50.2%	261,284	–
qwen38_confirm_v1	2,516,582,400	9.4%	65,536	–
qwen38_confirm_v2	2,516,582,400	9.4%	261,284	65,536

Table 1: Protocol and coverage for every run reported. Coverage and calibration differ between checkpoints, so cross-run differences are not attributable to scale alone.

- **No four-bit deployment claim.** A historical +0.008393 NLL four-bit measurement exists, but it is a 0.8B projection-subset result (0.24% of baseline NLL, 0.84% relative perplexity — distinct quantities). It is not a 27B result and not a full-coverage result. An earlier draft used it as both; that claim is withdrawn.
- **No causal scaling statement.** The two checkpoints differ in generation, coverage, tensor dimensions, calibration size and evaluated text extent. That the 27B margins are smaller is an observation, not an explained consequence of scale.
- **No equivalence results.** Where an interval includes zero we say *inconclusive*. We prespecified no equivalence margin, and failing to reject zero is not evidence of no effect.

4 Protocol

All arms share one protocol, and the properties below are the ones earlier versions of this work got wrong.

Storage is measured, not derived. Every codec serializes to packed bytes — five trits per byte, or c base- K codes per `uint64` with $K^c < 2^{64}$ — and bits per weight is computed from `len(bytes)`, not from $\log_2 K/d$. Multi-symbol packing makes the ternary and vector formats cost the same per index to within a few bytes per tensor, which is the conservative choice as well as the realistic one.

The evaluated weights are the stored weights. Group scales are rounded to FP16 *before* quantization, and every tensor is decoded back from its serialized bytes and required to match the quantizer’s reconstruction exactly before it is installed. In the confirmation run the artifact is additionally written to disk, reloaded and re-decoded. Omitting this check cost an earlier draft six points of headline: FP16 scale storage costs a 6561-point codebook +0.019 NLL while leaving a 3-point code unchanged.

Byte matching is near-equal, not literal. Where we say byte-matched, dimension-4 costs +0.0029% over scalar at 0.8B and +0.00012% at 27B; dimension-8 costs +0.43% and +0.0193%. Tables report five decimals.

Evaluation. Token-weighted NLL over the complete validation stream including the ragged final block, with a paired block bootstrap over ≥ 5000 resamples. Calibration uses the training split. Table 1 gives coverage and calibration size per run; they are not identical across runs, which is why cross-run differences are not attributed to scale.

offline, once per tensor

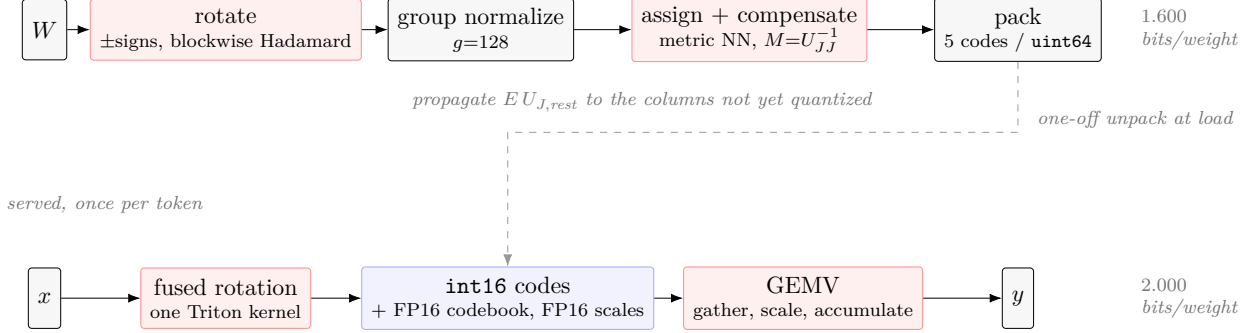


Figure 1: The recipe, and the two layouts it produces. **Offline**, each covered matrix is rotated by a blockwise randomized Hadamard transform, normalized in groups of 128, then quantized one column block at a time: within a block the assignment is a nearest-neighbour search under the metric $M = U_{JJ}^{-1}$, and the residual is propagated to the columns not yet quantized. That is GPTQ generalized to a vector code, and it collapses to textbook scalar GPTQ at dimension one, which we verify bit-exactly. **Served**, the stored packing is not what runs. Five base- K codes in a `uint64` give an optimal 1.600 bits of index but are inconvenient to address, so a one-off unpack at load time yields one `int16` code per group of weights, at 2.000 bits. The rotation moves to the *activation*, because the codes are stored in the rotated basis: $y = \text{rotate}(x) \widehat{W}^\top$. Quoting the storage rate as a resident footprint, as earlier drafts of this work did implicitly, overstates the served model by 25% on the index.

5 Method

Figure 1 shows the pipeline and the two layouts it produces.

Rotation is a blockwise randomized Hadamard transform on the input dimension. Codebooks are fitted by Lloyd/ k -means in group-normalized space and shared across all target tensors, charged once.

Extending GPTQ to a codec that quantizes d columns jointly is the only non-standard step. For a block of columns J the cost charged by the GPTQ derivation and the compensation applied to the remainder are

$$\|(W_J - Q_J) U_{JJ}^{-1}\|_F^2, \quad W_{\text{rest}} \leftarrow (W_J - Q_J) U_{JJ}^{-1} U_{J,\text{rest}},$$

with U the upper Cholesky factor of H^{-1} . The within-block decision is therefore a metric nearest-neighbour search with $M = U_{JJ}^{-1}$: transform points and codebook by M and search normally. At $d = 1$ this must reduce to textbook scalar GPTQ, and our test suite verifies it does so bit-exactly against an independently written reference. We regard this as a routine generalization.

6 Results

6.1 Dimension helps, and eight dimensions earn their place

At near-equal payload bytes the eight-dimensional code beats the learned scalar ternary code on both 0.8B tensor families and on the 27B projections (Table 2; the full arm-by-arm ablations are Tables 3, 4 and 5). Against a scalar control that spends 6.4% *more* bytes the margin is -0.034303

Contrast	Run	Δ NLL	95% CI	byte ratio
VQ8 vs scalar (0.8B)	0.8B/GPTQ	-0.036118	$[-0.038414, -0.033768]$	1.00430
VQ8 vs scalar (27B)	27B/frozen	-0.007121	$[-0.008513, -0.005725]$	1.00019
VQ4 vs scalar (0.8B)	0.8B/GPTQ	-0.024667	$[-0.027041, -0.022348]$	1.00003
VQ4 vs scalar (27B)	27B/frozen	-0.003087	$[-0.004364, -0.001807]$	1.00000
VQ8 vs scalar g64 (0.8B)	0.8B/GPTQ	-0.034303	$[-0.036660, -0.031971]$	0.93644
VQ8 vs scalar g64 (27B)	27B/frozen	-0.007836	$[-0.009247, -0.006356]$	0.93261
VQ8 vs VQ4 (27B)	27B/frozen	-0.004034	$[-0.005334, -0.002715]$	1.00019

Table 2: Primary and secondary contrasts, matched recipe across checkpoints. The 0.8B rows come from the GPTQ run, not the earlier uncompensated one. Each row names the run it comes from; rows from different runs are not matched comparisons. Byte ratios are printed to five decimals because “byte-matched” here means near-equal payload, not literal parity.

Arm	bits/wt	payload MB	Δ NLL vs BF16	95% CI
scalar3_rot	1.7250	24.42	+0.356599	$[+0.352028, +0.361286]$
scalar3_rot_actw	1.7250	24.42	+0.361084	$[+0.356485, +0.365764]$
scalar3_rot_gptq	1.7250	24.42	+0.113179	$[+0.109919, +0.116351]$
vq8_rot	1.7324	24.52	+0.240180	$[+0.236367, +0.243903]$
vq8_rot_actw	1.7324	24.52	+0.227827	$[+0.224049, +0.231700]$
vq8_rot_gptq	1.7324	24.52	+0.077061	$[+0.074362, +0.079868]$
vq8_rot_gptq_actwfit	1.7324	24.52	+0.077267	$[+0.074642, +0.079910]$
vq4_rot_gptq	1.7251	24.42	+0.088512	$[+0.085730, +0.091370]$
scalar3_g64_rot_gptq	1.8500	26.19	+0.111363	$[+0.108285, +0.114369]$
vq8_rot_gptq_seq	1.7324	24.52	+0.075171	$[+0.072454, +0.077936]$

Table 3: Codec and recipe ablation on the 0.8B DeltaNet QKV projections. Every arm is decoded from packed bytes and evaluated on the same stream, so the rows differ only in codec and recipe. Source: `results/ternary_task_v4` (0.8B, GPTQ recipe); BF16 NLL 3.436710.

$[-0.036660, -0.031971]$ at 0.8B under the matched compensated recipe; at 27B, over the complete validation stream, it is -0.007836 $[-0.009247, -0.006356]$ at a payload ratio of 0.93261 — the vector code wins while storing 6.7% *fewer* bytes. A 512-block pilot of the same comparison was inconclusive (-0.001736 $[-0.004444, +0.000923]$).

On the frozen confirmation, dimension-8 beats dimension-4 by -0.004034 with an interval of $[-0.005334, -0.002715]$ — more than half of dimension-8’s entire advantage over the scalar code, though on a single refit; at 0.8B the same contrast keeps its sign across all eight refits. The 512-block pilot had dimension-4 capturing 95.4% of the gain, with dimension-8 ahead by only 0.000181; that reading did not survive the complete stream. The difference is not sampling. The pilot’s blocks and the remaining 1,530 agree with each other, and BF16 loss on the pilot’s blocks is identical across runs to all printed digits. What differs is calibration size, 32,768 tokens against 65,536, which moved the storage-dominance contrast across disjoint intervals on the same evaluation data. Every interval reported here is conditional on one fitted quantizer, and calibration variability is not inside it.

6.2 Compensation dominates the codec, and inverts the proxy

Error compensation removes 68.3% of the damage on the 0.8B projections and 69.0% on the MLPs — both far larger than the codec effect. It achieves this while *raising* plain weight MSE by 38%

Arm	bits/wt	payload MB	Δ NLL vs BF16	95% CI
mlp_scalar3_rot	1.7250	56.98	+2.119720	[+2.104286, +2.134513]
mlp_scalar3_rot_gptq	1.7250	56.98	+0.658007	[+0.648510, +0.667584]
mlp_vq4_rot_gptq	1.7250	56.98	+0.552558	[+0.544087, +0.561040]
mlp_vq8_rot_gptq	1.7282	57.08	+0.500777	[+0.492819, +0.508608]
mlp_vq8_rot_gptq_pertype	1.7345	57.29	+0.521963	[+0.514011, +0.529926]
mlp_scalar3_g64_rot_gptq	1.8500	61.11	+0.656502	[+0.647276, +0.665713]

Table 4: The same recipe on all 72 MLP tensors of the 0.8B model. Every arm is decoded from packed bytes and evaluated on the same stream, so the rows differ only in codec and recipe. Source: `results/mlp_task_v1` (0.8B MLP, GPTQ recipe); BF16 NLL 3.436710.

Arm	bits/wt	payload MB	Δ NLL vs BF16	95% CI
scalar3_rot_gptq	1.7250	542.64	+0.010958	[+0.009366, +0.012561]
vq4_rot_gptq	1.7250	542.64	+0.007871	[+0.006436, +0.009331]
vq8_rot_gptq	1.7253	542.74	+0.003837	[+0.002420, +0.005250]
scalar3_g64_rot_gptq	1.8500	581.96	+0.011673	[+0.010078, +0.013264]

Table 5: Scale transfer, frozen confirmation: 48 DeltaNet QKV projections of Qwen3.8-27B over the complete validation stream. Every arm is decoded from packed bytes and evaluated on the same stream, so the rows differ only in codec and recipe. Source: `results/qwen38_confirm_v2` (27B, GPTQ recipe, frozen full stream); BF16 NLL 2.691117.

($7.18 \rightarrow 9.89 \times 10^{-5}$), so across recipes weight MSE is anti-predictive even though within a fixed recipe its ordering matches the loss ordering exactly. Diagonal activation weighting, by contrast, is worthless: harmful on scalar codes (+0.004485), and a tie once compensation is present (+0.000206), because the block metric already carries the diagonal and the cross terms.

6.3 Rate dominates compensation

Over 1.600 to 4.000 index bits at fixed 50.2% coverage, damage halves every 0.451, 0.506, 0.416, 0.530 and 0.468 bits — approximately exponential across a $2.40\times$ range of payload bytes, with no knee visible (Table 6). Six rates on one checkpoint at one coverage support “approximately exponential over this interval” and nothing stronger.

The four-bit point is measured on these same tensors: +0.018572 NLL, +1.87% perplexity. It is *not* interchangeable with the historical four-bit figure of +0.008393, which covers 15.1% of parameters rather than 50.2%.

Figure 2 plots the curve; the exponential fit and the absence of a knee are easier to see than to tabulate.

6.4 Coverage dominates rate, and cannot be exploited by ranking

MLP weights are $2.79\times$ more damaging per parameter than the projections. Given a fixed extra-byte budget, confining it to the MLP family beats spending it on the projections by -0.061105 [$-0.064430, -0.057804$].

The natural refinement — rank all 90 tensors by activation-weighted error removed per extra byte — *fails*. It promotes 14 of 18 projections and none of the 48 up/down projections, and loses to random selection of the same budget by +0.061244 [+0.058083, +0.064344]. Per parameter the

K	index bpw	total bpw	payload MB	Δ NLL	95% CI
81	1.600	1.7250	81.40	+0.637829	[+0.628446, +0.647323]
84	1.600	1.7250	81.40	+0.629249	[+0.619764, +0.638577]
255	2.000	2.1250	100.27	+0.340476	[+0.334386, +0.346756]
565	2.286	2.4108	113.76	+0.230274	[+0.225334, +0.235236]
1625	2.667	2.7920	131.74	+0.121974	[+0.118594, +0.125236]
7131	3.200	3.3262	156.95	+0.060716	[+0.058676, +0.062752]
65535	4.000	4.1361	195.17	+0.018572	[+0.017476, +0.019687]

Table 6: Uniform rate sweep at fixed coverage (50.2% of parameters), dimension 4 throughout so rate varies alone; six rates from 1.600 to 4.000 index bits. Stored-byte range $2.40\times$. Source: `results/rate_curve_v2`; BF16 NLL 3.436710.

Contrast	bootstrap half-width	range: draws	seeds	size*
VQ8 vs scalar	0.002416	0.004255	0.005057	0.004474
VQ8 vs scalar g64	0.002416	0.007078	0.005057	0.007110
VQ8 vs VQ4	0.002324	0.003893	0.006350	0.011189
g64 vs g128	0.001983	0.005548	0.000000	0.007441

Table 7: Conditional text-bootstrap half-width beside the range of each contrast over refits. These are different statistics and are not divided: a range over a few point estimates has no coverage interpretation. *Calibration size changes the experimental condition rather than resampling it. The zero for seed on the scalar-only contrast is a harness check: both arms use a deterministic Lloyd fit, so a seed cannot move it. Source: `results/calib_variability_v1`.

projections carry $5.7\times$ more activation-weighted error while causing $2.79\times$ less loss damage, so across families the proxy is anti-correlated with its target. The quantity measures error injected at a layer’s output and carries no information about propagation to the loss. This is the second forward-side ranking failure we record; a Fisher-based diagnostic gave Spearman $+0.015$.

6.5 How far the main result travels

Two studies probe the conditions under which the eight-dimensional advantage was measured, one varying the fit and the other the evaluation.

Refitting (Table 7). Eight refits over three disjoint calibration draws, three codebook seeds and four calibration sizes. All three primary contrasts keep their sign in 8/8 refits; VQ8 against scalar ranges from -0.037809 to -0.031061 . The ranges over refits are of the same order as, and in several cases larger than, the text-bootstrap half-width, so magnitudes are quoted as ranges rather than single margins. VQ8’s advantage grows monotonically with calibration size (-0.033335 at 16k tokens to -0.037809 at 131k), which is a condition effect rather than noise.

At 27B the ordering transfers on both axes — a second corpus (-0.008768) and 2048-token context (-0.005542), both excluding zero — but eight refits show the effect is smaller than the variability that refitting induces. The refit range is 0.007234 at 27B against 0.006748 at 0.8B, essentially the same absolute spread, while the mean effect is -0.003345 against -0.034095 . Seven of eight refits favour the vector code, four are individually significant, and one favours scalar. Refit variability does not shrink with scale; the effect does, so the 27B result is reported as a direction on average and the paper does not lean on it.

Varying the evaluation (Table 8). One fitted quantizer evaluated on the never-used wikitext-

Contrast	wt2_val_128	wt2_test_128	tiny_128	wt2_test_512	wt2_test_2048
VQ8 vs scalar	-0.036118	-0.038806	-0.043775	-0.035788	-0.031832
VQ8 vs scalar g64	-0.034303	-0.039543	-0.054717	-0.035518	-0.031779
VQ8 vs VQ4	-0.011451	-0.011840	-0.015677	-0.015220	-0.012270
g64 vs g128	-0.001815 [†]	+0.000737 [†]	+0.010942	-0.000270 [†]	-0.000053 [†]

Table 8: One fitted quantizer; only the evaluation varies. These are two corpora and three context lengths, not five independent replications — the three **wt2_test** columns share their text and differ only in blocking. [†] marks an interval including zero. Source: [results/generalization_v1](#).

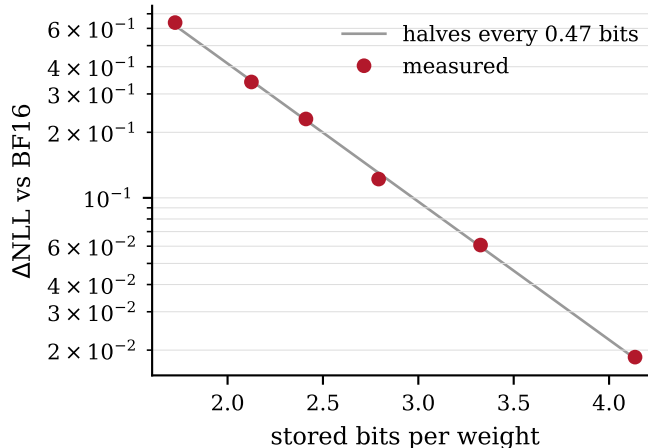


Figure 2: Damage falls approximately exponentially with stored rate, with no knee over the measured interval: seven codebook sizes from $K = 81$ to $K = 65535$ on the 90-tensor target set, halving about every 0.47 bits. A table of seven rows states this; a log axis shows it. Source: [results/rate_curve_v2](#).

2 test split, on TinyStories, and at 512 and 2048 token contexts. All three primary contrasts exclude zero in every configuration. Through 2048 tokens there is no ordering reversal and no large increase in aggregate damage, though VQ8’s advantage does shrink about 18% from the 128-token setting; aggregate loss averages over positions, so this is not evidence against recurrent-state drift. Out of domain, VQ8’s absolute advantage grows ($0.038806 \rightarrow 0.043775$) while the fraction of scalar damage it removes falls ($32.0\% \rightarrow 24.0\%$); we do not attribute the higher out-of-domain damage to calibration mismatch, which would need two calibration domains crossed with two evaluation domains.

The scalar group-size control has no stable sign: it flips across refits and, on TinyStories, excludes zero in the *unfavourable* direction ($+0.010942$). Section 7 resolves this into two opposed effects of similar size.

7 Calibration mismatch: an identity, a bound, and a crossed test

Changing the calibration corpus changes damage substantially while changing storage not at all, which makes it the sharpest lever in this study. The mechanism is visible in the objective the compensation step actually minimizes.

Calibration	code	wikitext-2 test	TinyStories	worst
wikitext	scalar3	+0.121115	+0.183486	+0.183486
wikitext	vq8	+0.082309	+0.140296	+0.140296
tinystories	scalar3	+0.288304	+0.036675	+0.288304
tinystories	vq8	+0.199492	+0.029201	+0.199492
mixed50	scalar3	+0.144351	+0.051790	+0.144351
mixed50	vq8	+0.089604	+0.042378	+0.089604

Table 9: Calibration domain crossed with evaluation domain, identical recipe, storage and evaluated text throughout, from `mechanism_v2` (65,536 calibration tokens in every condition). `mixed50` spends half its budget on each domain and is prespecified at 50/50; it is an exploratory intervention, designed after the pure conditions were seen. The last column is the worst-domain damage, which is what a mixture is meant to reduce.

7.1 Why mismatch has an effect at all

Write $E = \widehat{W} - W$ for the weight error a codec leaves behind. For a fixed input distribution D the layer reconstruction objective is quadratic,

$$L_D(E) = \mathbb{E}_D \|Ex\|_2^2 = \text{tr}(EH_DE^\top), \quad H_D = \mathbb{E}_D [xx^\top], \quad (1)$$

so evaluating on one domain a quantizer fitted on another costs exactly

$$L_{\text{test}}(E) - L_{\text{cal}}(E) = \text{tr}[E(H_{\text{test}} - H_{\text{cal}})E^\top], \quad (2)$$

and this is bounded elementarily by

$$|L_{\text{test}}(E) - L_{\text{cal}}(E)| \leq \|H_{\text{test}} - H_{\text{cal}}\|_2 \|E\|_F^2. \quad (3)$$

This is an identity and an elementary bound, not a theorem and not a statement about NLL. It says only that the penalty depends on how the error E is *oriented* relative to the difference of second moments: a pattern that is cheap under one domain’s activations can be expensive under another’s, and two quantizers with identical $\|E\|_F$ can differ arbitrarily in (2). Plain weight MSE is exactly $\|E\|_F^2$ up to a constant and therefore cannot see the difference at all — which is the same blindness that makes it invert against loss in our compensation and group-size results. Hessian-weighted reconstruction objectives of this form underpin GPTQ [3] and GPTVQ [1].

7.2 The crossed test

Two calibration corpora crossed with two evaluation corpora, identical recipe, storage and evaluated text throughout (Table 9).

Damage is minimized on the diagonal in all four cells, and the off-diagonal penalty is large: calibrating on TinyStories and evaluating on wikitext costs 0.288304 against 0.121115 matched. This rules out the alternative reading of our earlier out-of-domain result — that TinyStories is intrinsically harder for this recipe — since matched TinyStories costs 0.036675 against matched wikitext’s 0.121115, a factor of 3.3 the other way. It does *not* decompose the mismatched figure into a mismatch component and a sensitivity component; no design here does that. VQ8 wins in all four cells, removing 20.4% to 32.0% of scalar damage, with its proportional advantage largest where damage is largest.

Calibration	code	tokens	mean worst-domain	range over draws
wikitext	scalar3	65,536	0.190325	0.013840
TinyStories	scalar3	65,536	0.282648	0.014387
mixed 50/50	scalar3	65,536	0.143482	0.002791
wikitext, half budget	scalar3	32,768	0.196346	0.033205
TinyStories, half budget	scalar3	32,768	0.288453	0.014421
wikitext	vq8	65,536	0.147013	0.011311
TinyStories	vq8	65,536	0.191296	0.017389
mixed 50/50	vq8	65,536	0.095211	0.012307
wikitext, half budget	vq8	32,768	0.155934	0.021888
TinyStories, half budget	vq8	32,768	0.206481	0.023870

Table 10: Worst-domain damage by calibration condition, 3 disjoint calibration draws each, from `calib_mix_v2`. The half-budget rows are the control that separates domain coverage from sample count: for the vector code, halving the wikitext budget costs $+0.008921$ while spending that same halved budget on a second domain instead gains -0.060724 , so coverage is worth about seven times what sample count is worth here. The 50/50 split is prespecified, not tuned; the line of work is exploratory because the crossed cells were inspected first.

7.3 A fixed-budget calibration mixture

Equation (2) suggests an intervention. For the quadratic objective, $H_{\text{mix}} = \frac{1}{2}H_{\text{WT}} + \frac{1}{2}H_{\text{TS}}$ represents *exactly* the equally weighted average of the two domains’ reconstruction losses. Whether that improves worst-domain NLL does not follow — the objective is a local proxy and the mixture halves the data seen from each domain — so we measure it. The split is prespecified at 50/50 rather than tuned, the calibration budget is held at the same 65,536 tokens as every other condition, and each domain’s calibration text is disjoint from the text either is evaluated on. Because the crossed cells were inspected before this condition was designed, we report it as an *exploratory intervention*.

Over three disjoint calibration draws (Table 10) the mixture’s worst-domain damage is lower than either pure condition’s, for both codes, at identical storage: mean 0.095211 for VQ8 against 0.147013 and 0.191296, a 35% reduction against the better pure condition, and 0.143482 for scalar against 0.190325 and 0.282648 (25%). The price falls on the matched domain and is small — against the half-budget wikitext condition the mixture gives up 0.007311 on wikitext and gains 0.113104 on TinyStories, a sixteen-to-one trade, and all sixteen mixed-versus-pure contrasts keep their sign across all three draws.

This is domain coverage, not sample count. The mixture averages two 32,768-token Hessian estimates while each pure condition uses one 65,536-token estimate, so we ran the control: halving the wikitext budget costs $+0.008921$, while spending that same halved budget on a second domain instead gains -0.060724 . Coverage is worth about seven times what sample count is worth at this budget, and in the opposite direction. This is consistent with the mixture leaving a direction unprotected only when both domains do, though we inspected no directions and do not claim the mechanism is demonstrated.

Judged by this paper’s own yardstick, the gain is $4.2\times$ the mixture’s refit range for VQ8 and $16.8\times$ for scalar — the range of the 0.8B codec result ($5.05\times$), not the 27B one ($0.46\times$). This remains one model, one recipe, two corpora and one split; it does not establish that mixing is optimal or that 50/50 is the right weighting.

Contrast	Δ NLL	95% CI
grouping (g128 book fixed)	+0.014019	[+0.012133, +0.015908]
grouping (g64 book fixed)	+0.006568	[+0.004410, +0.008588]
codebook (g128 grouping fixed)	−0.005831	[−0.007809, −0.003792]
codebook (g64 grouping fixed)	−0.013281	[−0.015222, −0.011297]
<i>both at once (the diagonal)</i>	+0.000737	[−0.001213, +0.002710]
main effect, grouping	+0.010293	—
main effect, codebook	−0.009556	—
interaction	+0.007451	—

Table 11: The scalar group-size control is a 2×2 over grouping and codebook, not a single knob, from `mechanism_v2`. Moving to g64 at a fixed alphabet hurts; refitting the codebook in the new normalized space helps by a similar amount. An earlier draft read the diagonal as “group size alone” and concluded grouping was irrelevant; the two effects are of the same order and opposed, which is why the end-to-end contrast is small and unstable. Main effects and the interaction are differences of point estimates and carry no interval.

7.4 The group-size control, read as a factorial

The unstable group-size contrast has the same character. Moving from g128 to g64 changes the grouping *and* refits the shared codebook in the new normalized space, so the end-to-end contrast (+0.000737, [−0.001213, +0.002710]) is a diagonal of a 2×2 , not a main effect. An earlier draft read it as “group size alone moves nothing”; that reading is withdrawn. Read as the factorial it is (Table 11), all four single-factor contrasts exclude zero and only the diagonal does not: moving to g64 *hurts* by +0.014019 or +0.006568 depending on which alphabet is held fixed, and adopting the finer-fitted codebook helps by −0.005831 or −0.013281 depending on which grouping is held. Main effects are +0.010293 (grouping) and −0.009556 (codebook) with a +0.007451 interaction: two opposed effects of similar size, whose near-cancellation is what makes the end-to-end contrast small.

It does *not* explain why that contrast changes sign across refits, and we tested whether it does. Both codebooks here are deterministic Lloyd-Max fits over weights, so no draw or seed can move them and the Hessian is the only thing that varies across refits — which is also why the seed axis moved this contrast by exactly zero. Repeating the 2×2 over six refits (three disjoint draws, three calibration sizes), three of the four single-factor contrasts change sign themselves, and the one that varies most (grouping at a fixed g64 codebook, range 0.015179) varies more than the diagonal it was meant to explain (range 0.010206); the interaction excludes zero in none of the six. The cancellation account of the *instability* is therefore withdrawn. What the design does establish, since the codebooks are provably fixed, is that every effect in this family is a function of the compensation statistics alone, and that those statistics — estimated from 16k–131k calibration tokens — are noisy enough to move each contrast by 0.005 to 0.015, comparable to or larger than the contrasts themselves. We also note that the single-fit factorial above was evaluated on wikitext-2 test while the sign instability it was invoked to explain was measured on validation; the refit study is on validation and reproduces that study’s diagonal exactly.

That replacement account was itself given falsifiable predictions and tested against a reference Hessian from 524,288 tokens with four disjoint draws at each of four calibration sizes. Two hold: the Hessian’s relative deviation falls with calibration tokens at a log-log slope of −0.468 against the −0.5 of finite-sample noise, and the contrast’s within-size standard deviation falls at −0.494 — two exponents that have no reason to agree unless calibration sampling is what moves the contrast.

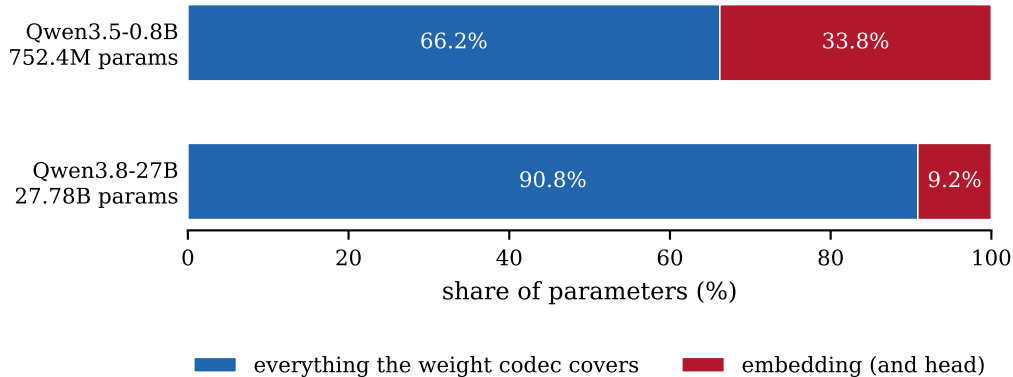


Figure 3: Where the parameters are. The weight codec in this paper — and in the low-rate literature generally — addresses the blue bar. At 0.8B the embedding is a third of the model and the codec cannot reach it; at 27B, where the head is untied and the embedding is charged twice, the same vocabulary occupies under a tenth. Norms and convolutions (0.07% at 0.8B) are folded into the blue bar. Counted from the checkpoint indices.

Arm	covered bpw	model MB	effective bpw	shrink	Δ NLL
r1600_k81	1.7250	616.9	6.559	2.44×	+1.000022 [+0.986485, +1.013520]
r2000_k255	2.1250	641.7	6.823	2.34×	+0.470715 [+0.462434, +0.478752]
r2667_k1625	2.7919	683.2	7.264	2.20×	+0.171490 [+0.167185, +0.175860]

Table 12: Every two-dimensional weight outside the embedding of Qwen3.5-0.8B: 186 tensors, 497,614,848 of 752,393,024 parameters (66.1%), from `fullmodel_v1`. **effective bpw** includes the BF16 embedding, which `tie_word_embeddings` also makes the output head.

The third fails: within a size, the correlation between a draw’s Hessian deviation and its contrast deviation is -0.142 over 16 draws, i.e. none. Equation (2) explains why we should not have expected otherwise — the penalty depends on how the Hessian error is *oriented* relative to the weight error, and a Frobenius norm is orientation-blind. We therefore claim only what was tested: the spread scales like finite-sample noise, and no scalar measure of Hessian error predicts an individual refit. One further observation was not predicted by anything: all four draws at 65,536 tokens give a negative contrast and all four at 131,072 a positive one, so calibration size shifts this quantity’s sign rather than merely adding noise to it.

8 Full coverage and a compressed matmul

8.1 Converting every non-embedding matrix

All 186 two-dimensional weights outside the embedding, 497,614,848 of 752,393,024 parameters (66.1%), evaluated on the complete validation stream (Table 12). At ternary rate this costs $+1.000022$ NLL $[+0.986485, +1.013520]$ — perplexity nearly triples — against $+0.470715$ at 2.125 bits and $+0.171490$ at 2.792 bits.

Damage across disjoint tensor families is *approximately* additive at this rate (Table 13), and we measured the residual directly. The observed totals agree with an additive reference to within 0.5%, but those rows come from separate runs that each fit a codebook on their own tensor pool, so the

Run	target set	params	bpw	ΔNLL	avg. per 100M
ternary_task_v4	in_proj_qkv (18)	113,246,208	1.7251	+0.088512	0.078
mlp_task_v1	MLP (72)	264,241,152	1.7250	+0.552558	0.209
mixed_alloc_v1	both (90)	377,487,360	1.7250	+0.637829	0.169
fullmodel_v1	all non-embedding (186)	497,614,848	1.7250	+1.000022	0.201

Table 13: Damage against coverage at ternary rate. Every row is asserted at generation time to share the same BF16 baseline to full double precision (3.436710) and the same target count (261,284); a matching count alone would not establish stream identity, but a loss computed over the whole stream agreeing in all digits does. The two disjoint families give an apparent additive reference, $0.088512 + 0.552558 = 0.641069$ against a measured 0.637829, but these rows fit their codebooks on different pools; the frozen-artifact test reports an interaction of +0.003818 [+0.000769, +0.006835], so damage is slightly super-additive. The jump at full coverage is composition, not breadth: going from the 90-tensor set to all 186 adds 120,127,488 parameters for +0.362193, i.e. 0.302 per 100M, the highest marginal cost in the model. The last column is an average over each row’s own target set, not a marginal rate.

Arm	tensors	params	ΔNLL	95% CI
A — in_proj_qkv	18	113,246,208	+0.085350	[+0.082522, +0.088216]
B — MLP	72	264,241,152	+0.548660	[+0.540311, +0.557142]
AB — both	90	377,487,360	+0.637829	[+0.628446, +0.647323]
interaction			+0.003818	[+0.000769, +0.006835]

Table 14: Does coverage damage compose additively? One Hessian capture, one codebook fitted over all 90 tensors and one quantization pass at 1.7250 bits/weight, then the dequantized weights installed in three combinations, so every tensor carries bit-identical weights in every arm that includes it. The interaction is $L_{AB} - L_A - L_B + L_{\text{BF16}}$, bootstrapped over the same resampled blocks. It excludes zero, so damage is slightly *super*-additive — though at 0.003818 against a total of 0.637829 it is 0.6% of the effect, so “approximately additive” remains a fair description. From `interaction_v1`.

comparison confounds a family interaction with a change in the codec each family receives. Quantizing once, freezing the artifacts and installing subsets removes that confound: with bit-identical weights in every arm that includes a tensor, QKV alone costs +0.085350, MLP alone +0.548660, both +0.637829, and the interaction $L_{AB} - L_A - L_B + L_{\text{BF16}}$ is +0.003818 [+0.000769, +0.006835] — small, but excluding zero, so damage is slightly *super*-additive (Table 14). An earlier draft called damage “markedly super-linear in coverage” (wrong by orders of magnitude) and a later one called it “additive” (wrong in sign); the measured answer is a small positive interaction. The jump to full coverage is largely composition, though we cannot attribute all of it to the added 120,127,488 parameters, since the full-coverage run also refits its codebook over a four-times-larger pool.

The byte accounting is the more consequential result. The covered matrices compress $9.28\times$ ($995 \rightarrow 107$ MB), but the model shrinks only $2.44\times$: the embedding, which `tie_word_embeddings` also makes the output head, stays at BF16 and is then 83% of what remains, putting the effective rate at 6.559 bits/weight while the covered tensors sit at 1.725. **At these rates the binding constraint is embedding precision, not the weight codec** (Figure 3), and crossing the two axes shows the exchange rate is not close (Table 15). The embedding quantizer is deliberately naive — per-row or per-group scaling, no rotation, no compensation — and still, in ΔNLL per 100 MB

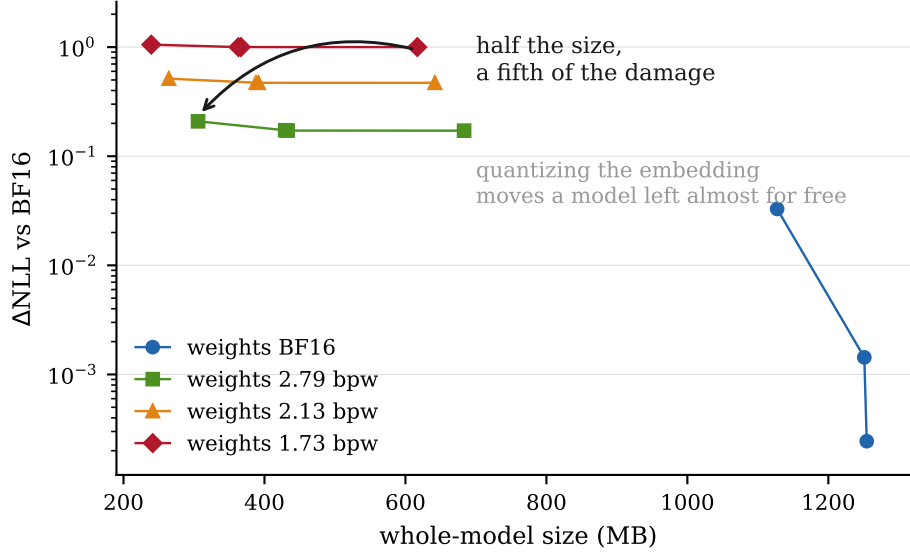


Figure 4: Whole-model size against damage, over the weight-rate $\times$ embedding-precision grid. Each series fixes a weight rate and varies the embedding from BF16 (rightmost point) to INT4 (leftmost), so moving left along a series is spending nothing but embedding precision. The sub-two-bit series sit above and to the right of points that are both smaller and better: the dominance the text describes is the arrow. The BF16/BF16 reference has exactly zero damage and no position on a log axis; it is 1504.8 MB. Source: `results/embed_precision_v1`.

saved, moving it from BF16 to INT8 costs 0.000098 against 0.020873 for the first step of the weight codec and 2.127315 for its last. The conjecture that FP8 on the embedding would save more bytes than moving the covered tensors from 2.792 to 1.725 bits, at far less cost, holds with room to spare: 253.8 MB against 66.4 MB, at +0.001434 against +0.828532.

The consequence is a dominance rather than a trade-off (Figure 4). A model with 2.792-bit weights and an INT4 embedding is 305.8 MB at +0.208926; one with ternary weights and a BF16 embedding is 616.9 MB at +1.000022 — half the size and a fifth of the damage. *On this model, every sub-two-bit configuration we measured is off the efficient frontier for whole-model size.* This does not weaken the codec comparisons, which hold coverage and bytes fixed; it withdraws the implicit premise that driving the weight rate down is how to make this model smaller. It is also a statement about parameter shares rather than about codecs: the embedding is 33.8% of this checkpoint on a 248,320-token vocabulary, while on the 27B checkpoint — same vocabulary, hidden size 5120, untied head — embedding and head together are 2,542,796,800 of 27,781,427,952 parameters, 9.15% counted from the checkpoint index, and the arithmetic inverts. Nothing here measures the embedding’s quality cost at 27B, so that inversion is an argument about bytes and not a result.

8.2 A compressed matmul on a real projection

Storage layout is not runtime layout: five base-6561 codes in a `uint64` give an optimal 1.600 bits of index but are inconvenient to index, since $6561^5 > 2^{63}$ sets the sign bit. PyTorch will hold a `uint64` tensor but raises `NotImplementedError` on its arithmetic, so the host cannot do the digit extraction; Triton does have `tl.uint64`, and there the extraction costs five integer divisions per word on the critical path. One code per `int16` costs 2.000 bits instead. Nor is the runtime

Weights	Embedding	model MB	eff. bpw	shrink	ΔNLL
bf16	bf16	1504.8	16.000	1.00×	+0.000000
bf16	fp8_e4m3	1251.0	13.302	1.20×	+0.001434
bf16	int8_g128	1254.5	13.339	1.20×	+0.000244
bf16	int4_g128	1127.3	11.987	1.33×	+0.032871
r1600_k81	bf16	616.9	6.559	2.44×	+1.000022
r1600_k81	fp8_e4m3	363.1	3.860	4.14×	+1.000291
r1600_k81	int8_g128	366.5	3.897	4.11×	+1.000814
r1600_k81	int4_g128	239.4	2.546	6.29×	+1.053793
r2000_k255	bf16	641.7	6.823	2.34×	+0.470715
r2000_k255	fp8_e4m3	388.0	4.125	3.88×	+0.471673
r2000_k255	int8_g128	391.4	4.162	3.84×	+0.470934
r2000_k255	int4_g128	264.3	2.810	5.69×	+0.514113
r2667_k1625	bf16	683.2	7.264	2.20×	+0.171490
r2667_k1625	fp8_e4m3	429.4	4.566	3.50×	+0.172940
r2667_k1625	int8_g128	432.9	4.603	3.48×	+0.171893
r2667_k1625	int4_g128	305.8	3.251	4.92×	+0.208926

Table 15: Where a byte is best spent. The embedding is 254,279,680 parameters, tied to the output head, so quantizing it perturbs both the input lookup and every logit; its quantizer here is deliberately naive (per-row or per-group scaling, no rotation, no compensation), which makes the comparison conservative against the weight codec. Marginal cost in ΔNLL per 100 MB saved: 0.000098 for BF16→INT8 on the embedding against 0.020873 for BF16→2.792 bits on the weights and 2.127315 for 2.125→1.725 bits. Note the dominance: r2667_k1625|int4_g128 is 305.8 MB at +0.208926 against r1600_k81|bf16 at 616.9 MB and +1.000022 — half the size and a fifth of the damage. Source: `embed_precision_v1`.

activation the stored one: the codes hold $\text{rotate}(W)$, so a compressed path must apply the same blockwise Hadamard to its input, $y = \text{rotate}(x) \widehat{W}^\top$.

On `in_proj_qkv` of the 0.8B model quantized with the full recipe, and on activations captured from the model, the Triton path’s relative error against an exact FP32 product is 2.71×10^{-7} , against 3.55×10^{-3} for the installed BF16 weights that every ΔNLL in this paper was measured with (all paths scored on the same 256 rows with the same denominator). *The compressed kernel closely reproduces the quantized FP32 reference on the tested activations; its end-to-end quality relative to decoded BF16 execution remains unmeasured.* An earlier draft called the reported NLL figures “conservative” on this evidence; that does not follow, since BF16 rounding can reinforce or partially cancel quantization error and the nonlinear layers downstream need not respond monotonically to a local perturbation.

Resident bytes fall 7.08× on that tensor and 7.47× at the 27B QKV shape, the difference being codebook amortization; peak allocation during a product falls 5.00× (113.5 → 22.7 MB). Latency does not improve, though it is far better than our first implementation suggested. With the rotation written as ten Python-controlled butterfly stages the compressed path took 0.5233 ms against 0.0187 ms for cuBLAS, 0.4760 ms of it in the transform. That measured the prototype, not the transform: the radix-2 FWHT in natural order is a Kronecker product, so a 1024-point block viewed as a 32×32 tile X satisfies $\text{FWHT}(x) = \text{vec}(H_{32} X H_{32})/32$ — two small matrix products in one kernel with the sign flip folded in. Fused, the rotation costs 0.0182 ms (26.1× faster, bit-identical to the reference on this input) and the whole compressed path 0.0555 ms, or

Quantity	BF16	served compressed
NLL over 32,768 targets	3.398479	3.398463
resident (allocator)	1504.8 MB	783.2 MB
decode step	85.6 ms	99.4 ms

Table 16: All 186 non-embedding matrices replaced by modules holding only int16 codes, an FP16 codebook and FP16 group scales, with the rotation fused; the dense weights are dropped, so what is reported here is what the allocator and the evaluation loop actually saw. The served model reproduces the decoded-BF16 loss to $-1.59e - 05$ (CI $[-0.000458, +0.000428]$), so it is the model the rest of this paper measured. Resident bytes come from `torch.cuda.memory_allocated`, and the $1.92\times$ is below the $2.20\times$ derived from payload counts at this rate because the served layout spends 2.000 bits of index where the file spends 1.600. Decode figures are dominated by an unfused recurrent path in both arms. Source: `endtoend_v1`.

$3.0\times$ cuBLAS rather than $28\times$. That agrees with the $3.1\times$ GEMV-only comparison at the larger synthetic shape, where no transform is in the loop. The format is verified, smaller, and about three times slower at batch one; neither kernel is tuned beyond correctness.

8.3 A model that runs on compressed weights

Every quality figure above was produced by decoding packed bytes into BF16 weights and installing those. Replacing all 186 non-embedding matrices with modules that hold only int16 codes, an FP16 codebook and FP16 group scales — dense weights dropped, rotation fused — gives a model that runs (Table 16). It reproduces the decoded-BF16 loss to -1.59×10^{-5} , well inside the evaluation’s own noise, so the served model is the model this paper measured. Its resident footprint, taken from the allocator rather than from a payload count, is 783.2 MB against 1504.8, a factor of 1.92 — *below* the $2.20\times$ the byte arithmetic gives at this rate, because the served layout spends 2.000 bits of index where the file spends 1.600. A single decode step costs 99.4 ms against 85.6 ms. Both figures are dominated by an unfused recurrent path — the installed library reports that the fast linear-attention kernel is unavailable, and 85.6 ms per token for a 0.8B model is not a serving number — so the $1.16\times$ ratio is the increment compressed weights add to an unoptimized loop and nothing more. We do not report a decode-throughput claim; the interpretable latency measurement is the isolated projection above.

The embedding, norms, KV cache and recurrent state remain BF16 and both kernels are tuned only far enough to be correct. What this settles is narrow and was previously open: the byte counts in this paper describe a model that ran.

9 Negative results retained from earlier work

Two hypotheses were pursued at length and refuted; they are retained because their failure mode recurs.

Polar weight codes. Storing a magnitude and a phase index per weight pair appeared to beat an equal-byte real four-bit code. It beat only a control with a *uniform* codebook; giving the scalar control the same Lloyd–Max fitting reverses the ranking. Repairing the polar assignment changes MSE by 0.035%, because with five phase bits $1 - \cos(\pi/32) \leq 4.8 \times 10^{-3}$ while one radius-grid step is $\approx 14\%$ of the row scale.

Self-consistent calibration. Solving activation-aware calibration as a DFT-style fixed point

is unnecessary: the forward-only dependency graph is acyclic, so one ordered pass is exact, and closed-form one-pass corrections already exist [6]. Measured, sequential capture is worth -0.001890 $[-0.003421, -0.000315]$ at $3.5\times$ the cost.

The pattern. Both, and the ranking failure above, are the same error: a method beat a control that had been given less tuning, or a proxy was trusted outside the regime where it was fitted. Every correction in this paper came from a control — a fitted scalar baseline, a random allocation, a bit-exactness assertion — and not from inspection.

10 Reproducibility and limitations

Runs record a manifest: arguments, source hashes, checkpoint hashes, library and CUDA versions, device map, GPU budget, calibration size, and the evaluated versus available target counts. Tables in this paper are generated directly from saved results by `experiments/make_tables_v2.py`; an earlier draft mixed an uncompensated margin into a compensated comparison, and generation is how we prevent recurrence rather than by care.

An exactness failure, observed once and not reproduced. One run at a 17 GiB GPU weight budget raised the per-tensor bit-exactness check with a decoded-versus-reference difference of 0.036. Seven subsequent runs at that budget, across two script versions in fresh processes, instead terminated with an identical out-of-memory error before reaching that state; 15 and 16 GiB are exact. The corruption was never reproduced and no failing tensor was captured, so it could not be localized. One observation of a wrong value stands against seven of a clean failure, and an earlier draft’s framing of this as a characterized budget-sensitive defect is withdrawn. Every reported run used 15 GiB, every tensor passed the strict check, and the confirmation run additionally re-decodes each artifact from disk.

Statistical scope. Intervals measure text-sampling variability conditional on one fitted quantizer, and that conditioning is load-bearing. Refitting over three calibration draws and three codebook seeds produces ranges of the same order as, and in several cases larger than, the text-bootstrap half-width; calibration size is reported separately again, as a condition change rather than a further sample. We report the two statistics side by side and do not divide them: a range over a handful of refits and a bootstrap half-width are different quantities, and their ratio has no coverage interpretation. Directions survive — the vector code wins in all eight refits on all three primary contrasts — but a single six-decimal margin from one refit is more precise than the procedure warrants, so magnitudes are quoted as ranges (-0.031 to -0.038 for VQ8 against scalar). The scalar group-size contrast changes sign across refits and across evaluation domains, and the factorial in Section 7 shows why: grouping and codebook are opposed effects of similar size, so their end-to-end difference is small and unstable. The 27B intervals carry the same conditioning; refit variability *is* now measured there (Section 6) and is larger than the effect.

Generalization scope. The five evaluation configurations in Section 6 are two corpora and three context lengths, not five independent replications; the three wikitext-test rows share their text. Position-resolved loss is measured with per-block intervals over 127 blocks, and we report it as weak evidence and lean on it nowhere. An earlier draft said VQ8 damage “rises monotonically” across a 2048-token block; with intervals the endpoint contrast does not exclude zero and the rise is not monotone. What erodes is VQ8’s *advantage* over the scalar code, $+0.009695$ from the first bucket to the last, and that excludes zero on all five bootstrap seeds at 100,000 replicates — but by a lower bound between 5×10^{-5} and 1.3×10^{-4} , two orders of magnitude below the effect, and the analysis followed inspection of the bucket means rather than preceding it. Scalar damage shows no trend. Position and token content are confounded within a block in any case, so we report position-

dependent damage and do not identify recurrent-state drift as its mechanism; that would need the same tokens evaluated with and without a state reset, which was not run. Higher damage out of domain is attributed to calibration mismatch on the strength of the crossed design in Section 7, which rules out the alternative reading that TinyStories is intrinsically harder for this recipe; it does not decompose the mismatched figure into mismatch and sensitivity components.

Availability

A single generator script produces every table in this paper directly from the saved run artifacts. It reads the JSON each run wrote, and it refuses to emit a table whose sources disagree on the BF16 baseline or on the evaluation stream. The runners, the saved results and the running record of the study — including the sections withdrawn or corrected along the way — are at <https://github.com/shayan65/low-rate-quantization>.

References

- [1] GPTVQ: The Blessing of Dimensionality for LLM Quantization. <https://arxiv.org/abs/2402.15319>
- [2] QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. <https://arxiv.org/abs/2402.04396>
- [3] GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. <https://arxiv.org/abs/2210.17323>
- [4] QuIP: 2-Bit Quantization of Large Language Models With Guarantees. <https://arxiv.org/abs/2307.13304>
- [5] Extreme Compression of Large Language Models via Additive Quantization. <https://arxiv.org/abs/2401.06118>
- [6] GPTAQ: Efficient Finetuning-Free Quantization for Asymmetric Calibration. <https://arxiv.org/abs/2504.02692>
- [7] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefer and J. Hensman. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. <https://arxiv.org/abs/2404.00456>
- [8] M. Williams and N. Aletras. On the Impact of Calibration Data in Post-training Quantization and Pruning. <https://arxiv.org/abs/2311.09755>